

Лекция 1

Введение в Web и Шаблонизация

Проектирование систем и продуктовая веб-разработка

Канев Антон Игоревич

Канев Антон Игоревич



Окончил МГТУ им. Н.Э.
Баумана в 2016 г.

Аспирантуру МГТУ им. Н.Э.
Баумана в 2020 г.

Окончил магистратуру
университет Glyndwr
(Рексем, Великобритания)



Курс по глубокому
обучению

Курс по web разработке

English	Advanced
Español	B2
中文	HSK4



NVIDIA DLI Certificate

РосЕвроБанк
Совкомбанк

Руководитель проектов



Московский государственный
технический университет
имени Н.Э. Баумана

Оценивание и сроки

- Экзамен
- 2 рубежных контроля
- Практические задания – закрепление и использование знаний разных дисциплин
- Оценивание – баллы за задания
- Сроки!!!
- Участие в Хакатоне ИУ5 2026 (~конец октября)

* Одна тема на весь курс

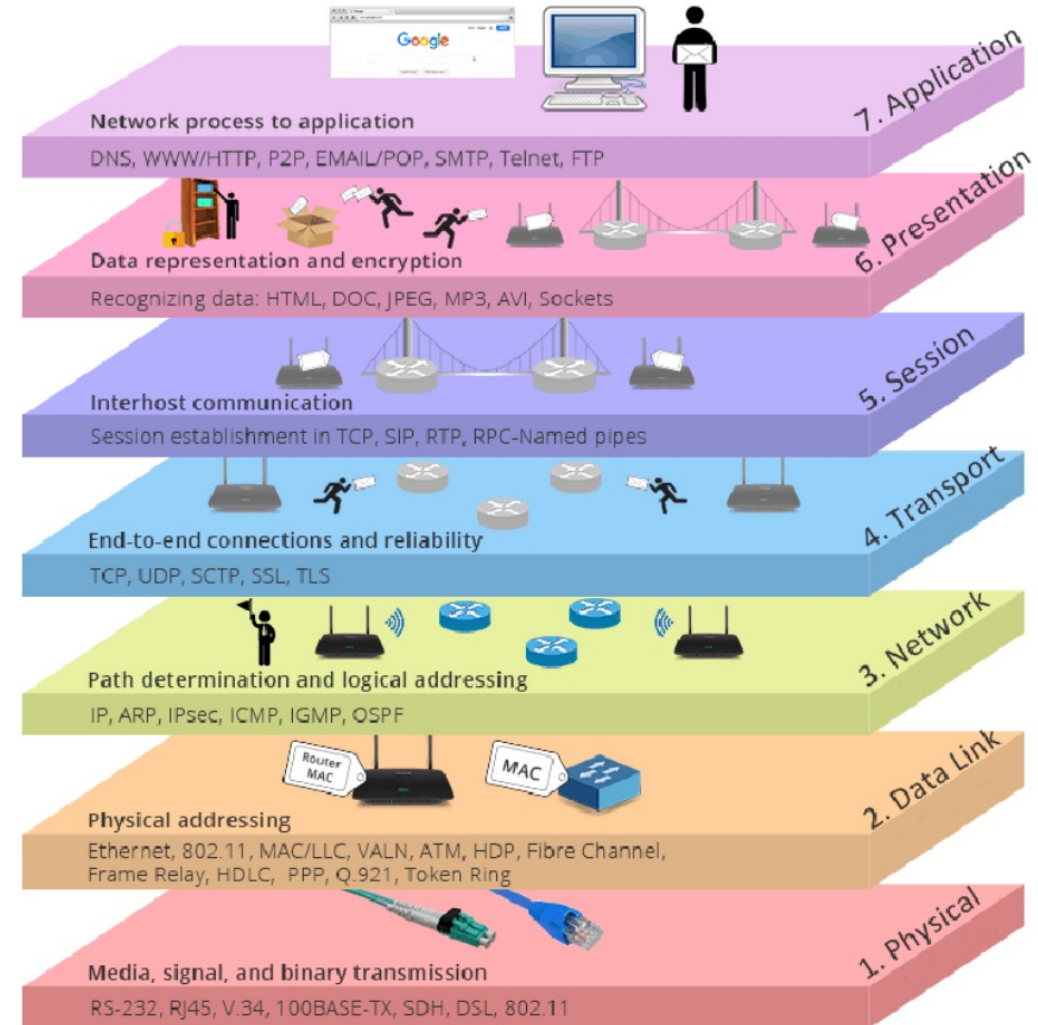
- Набор требований по каждому заданию + порядок показа
- Все названия таблиц, файлов, классов, страниц, url и тд называть строго по теме
- 8 лабораторных + GitHub + UML + защита + конспект
- ТЗ (Модуль 1)
- ДЗ: три дополнительных задания и готовый Отчет-РПЗ **TestVKR** + распечатать **статью** [Arxiv.org](https://arxiv.org) и **перевести** термины
- Знание браузера, умение использовать необходимые инструменты. Знание своего кода для **письменной защиты**
- Ответы на вопросы по базовым понятиям и технологиям

*Стек технологий

- React (самый популярный в РФ и мире) + Redux + React Bootstrap
- Fast API, NestJS или Go. Другой бэкенд только по согласованию с преподавателем
- PostgreSQL
- GitHub - репозитории для фронтенда, бэкенда, нативного приложения. Вы работаете на свое портфолио
- VS Code – основная среда разработки
- Docker Compose

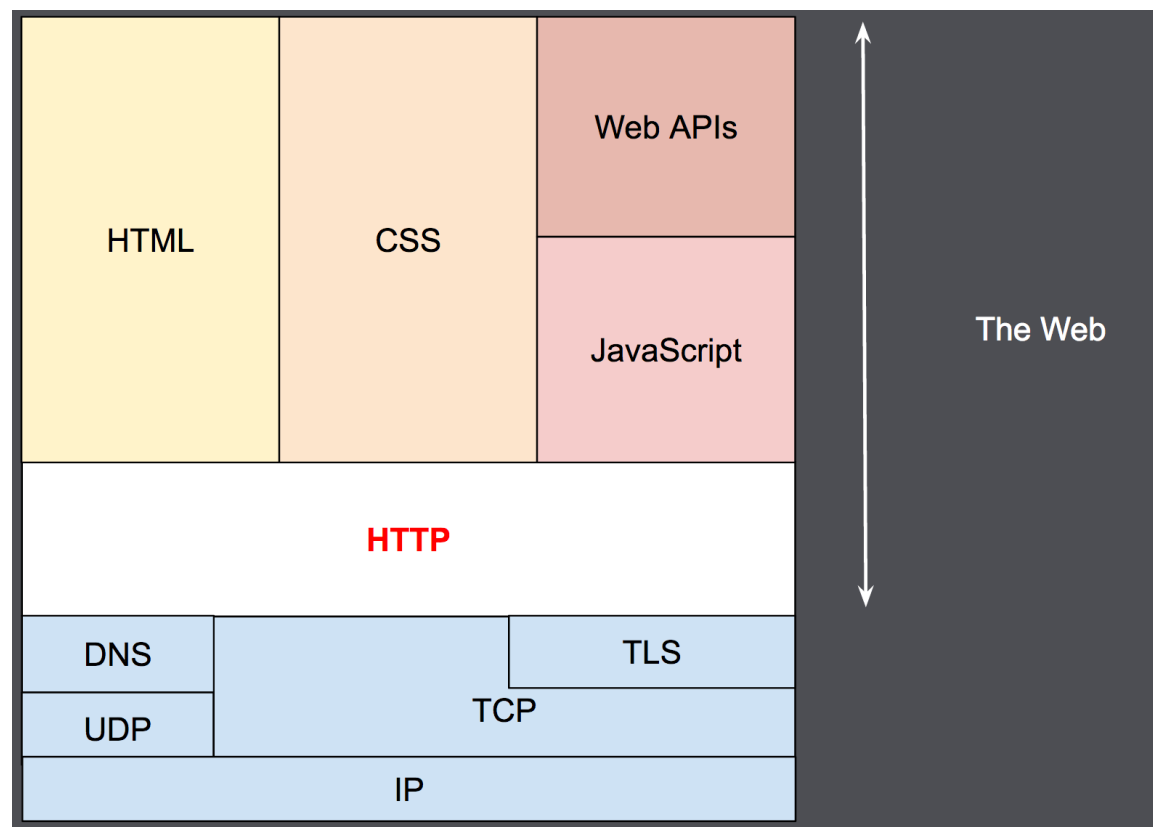
*Компьютерные сети. Модель OSI

- 7-ми уровневая модель OSI
- Это детальное описание интернета
- Приложения работают на самом высоком 7-ом уровне
- Порты для программ на 4-ом
- IP адреса компьютеров 3 уровень
- Физическая среда передачи на первом уровне



*Граница Web/интернет

- Стандарты Web публикуются на сайте веб-консорциума
- <https://www.w3.org>



*Компоненты Web – знать обязательно!

- Тим Бернерс-Ли создал три основных компонента WWW:
- язык гипертекстовой разметки документов HTML (HyperText Markup Language);
- универсальный способ адресации ресурсов URI (Universal [Uniform] Resource Identifier);
- протокол обмена гипертекстовой информацией HTTP (HyperText Transfer Protocol – протокол передачи гипертекста).
- Позже к этим **трем** компонентам добавился **четвертый** CGI: исполняемая часть, с помощью которой можно создавать динамические HTML-документы. В нашем курсе мы будем называть это Server-Side Rendering **SSR** (и как разновидность **шаблонизация**)

HTML

- HTML-HyperText Markup Language.
- В HTML версии 1.0 были реализованы все элементы разметки, связанные с выделением параграфов, шрифтов, стилей и т.п., т.к. уже первая реализация подразумевала графический интерфейс. Важным компонентом языка стало описание гипертекстовых ссылок, графики и обеспечение возможности поиска по ключевым словам.
- В качестве базы для разработки языка гипертекстовой разметки HTML был выбран SGML (Standard Generalised Markup Language – стандартный общий язык разметки). Тим Бернерс-Ли описал HTML в терминах SGML как описывают языки программирования в терминах формы Бекуса-Наура.

*URI – схема HTTP

- http:// хост : порт / путь и имя файла ? параметры # якорь гиперссылки

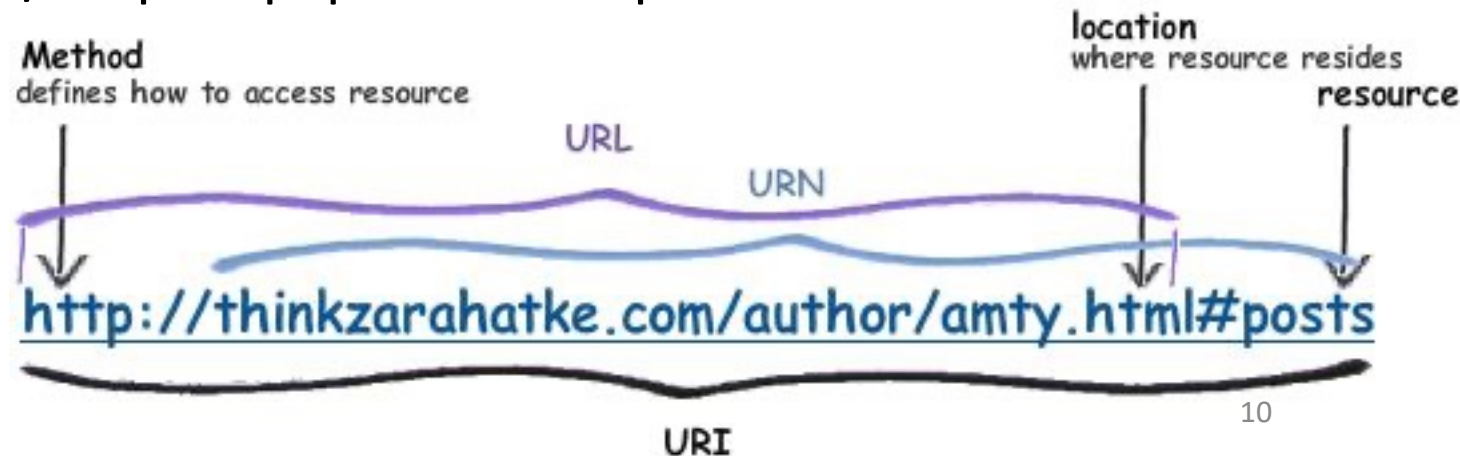
Пример:

http:// 127.0.0.1 :8080/index.html

http://localhost:8080/file.html

http://iu5.bmstu.ru:8080/cat1/cat2/script.asp?param1=1¶m2=2#anchor1

- Порт по умолчанию – 80.



* HTTP request/response

- Методы

GET, POST, PUT, ...

- Коды состояний

200 OK

404 Not Found

- Заголовки

параметр: значение

```
File Edit View Search Terminal Help
[osboxes@osboxes ~]$ telnet iu5.bmstu.ru 80
Trying 195.19.50.252...
Connected to iu5.bmstu.ru.
Escape character is '^]'.
GET / HTTP/1.0

HTTP/1.1 200 OK
Server: nginx
Date: Mon, 09 Nov 2020 08:53:01 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 985
Connection: close
Last-Modified: Fri, 12 Apr 2019 09:22:18 GMT
ETag: "3d9-58651d6d73b52"
Accept-Ranges: bytes

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" lang="en"><head>
  <title>hoster1.uimp.bmstu.ru &mdash; Coming Soon</title>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8"/>
  <meta name="description" content="This is a default index page for a new domain."/>
  <style type="text/css">
    body {font-size:10px; color:#777777; font-family:arial; text-align:center;}
    h1 {font-size:64px; color:#555555; margin: 70px 0 50px 0;}
    p {width:320px; text-align:center; margin-left:auto;margin-right:auto; margin-top: 30px }
    div {width:320px; text-align:center; margin-left:auto;margin-right:auto;}
    a:link {color: #34536A;}
    a:visited {color: #34536A;}
    a:active {color: #34536A;}
    a:hover {color: #34536A;}
  </style>
</head>
```

HTTP запросы в браузере

- На вкладке Network отображаются все запросы
- Получение HTML, js, css, изображений, а также AJAX (Fetch/XHR) запросы
- Отображаются заголовки, ответ и тд

The screenshot shows the Chrome DevTools Network tab. The top panel displays a list of requests, with 'yandex.com' selected. The bottom panel shows the details for this request, including the Request URL, Request Method, Status Code, Remote Address, Referrer Policy, and Response Headers.

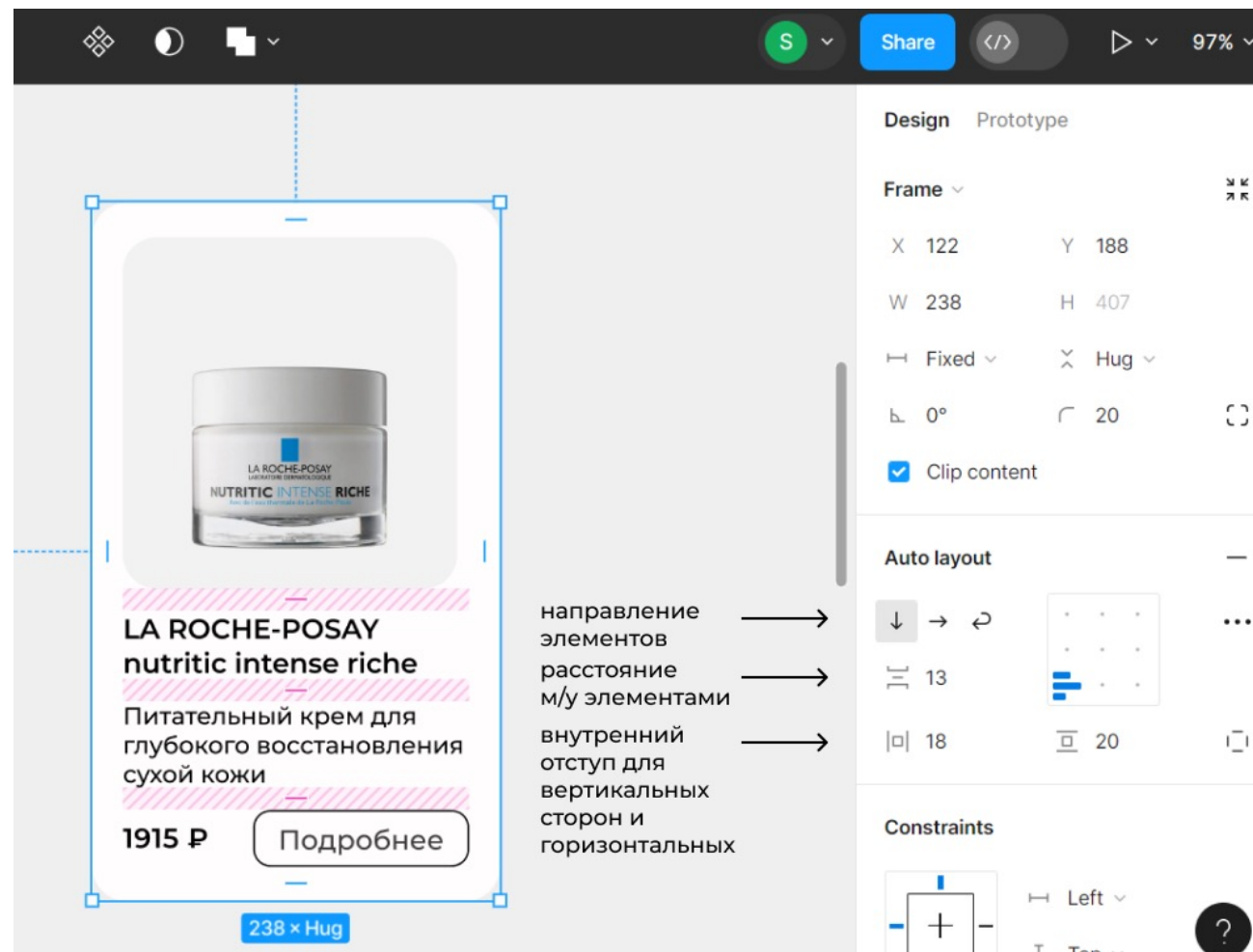
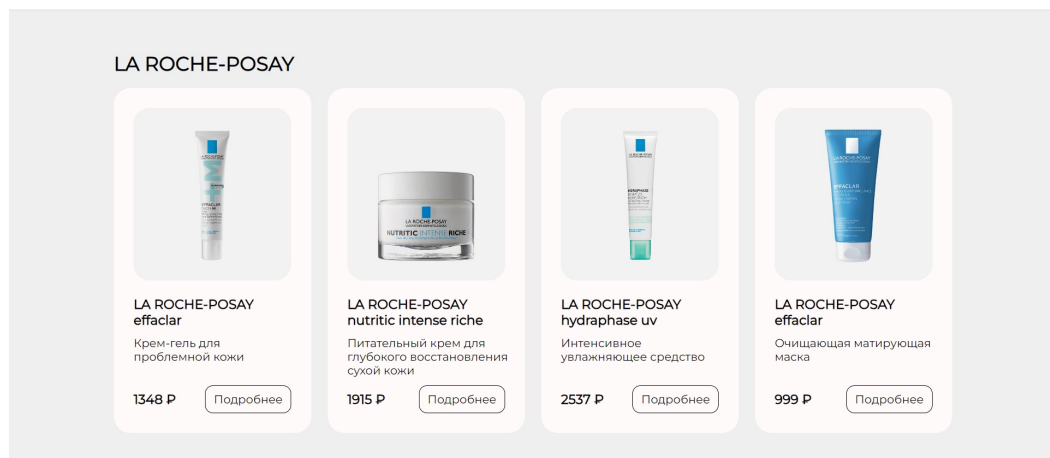
Name	Headers	Preview	Response	Initiator	Timing	Cookies
yandex.com	General					
AtLYSQcGuvVjj9g_BOHYQ3r...	General					
text-regular.woff2	General					
text-medium.woff2	General					
data:image/svg+xml;...	General					
games_new_v2.2.png	General					
images0.2.png	General					
video0.2.png	General					
meteum_v2.2.png	General					
maps2.2.png	General					
skc_n.svg	General					
watch_serp.js	General					

General	
Request URL:	https://yandex.com/
Request Method:	GET
Status Code:	200
Remote Address:	77.88.55.80:443
Referrer Policy:	origin

Response Headers	
Accept-Ch:	Sec-CH-UA-Platform-Version, Sec-CH-UA-Mobile, S UA-WoW64, Sec-CH-UA-Arch, Sec-CH-UA-Bitness, Device-Memory, RTT, Dnlink, ECT
Cache-Control:	no-cache,no-store,max-age=0,must-revalidate

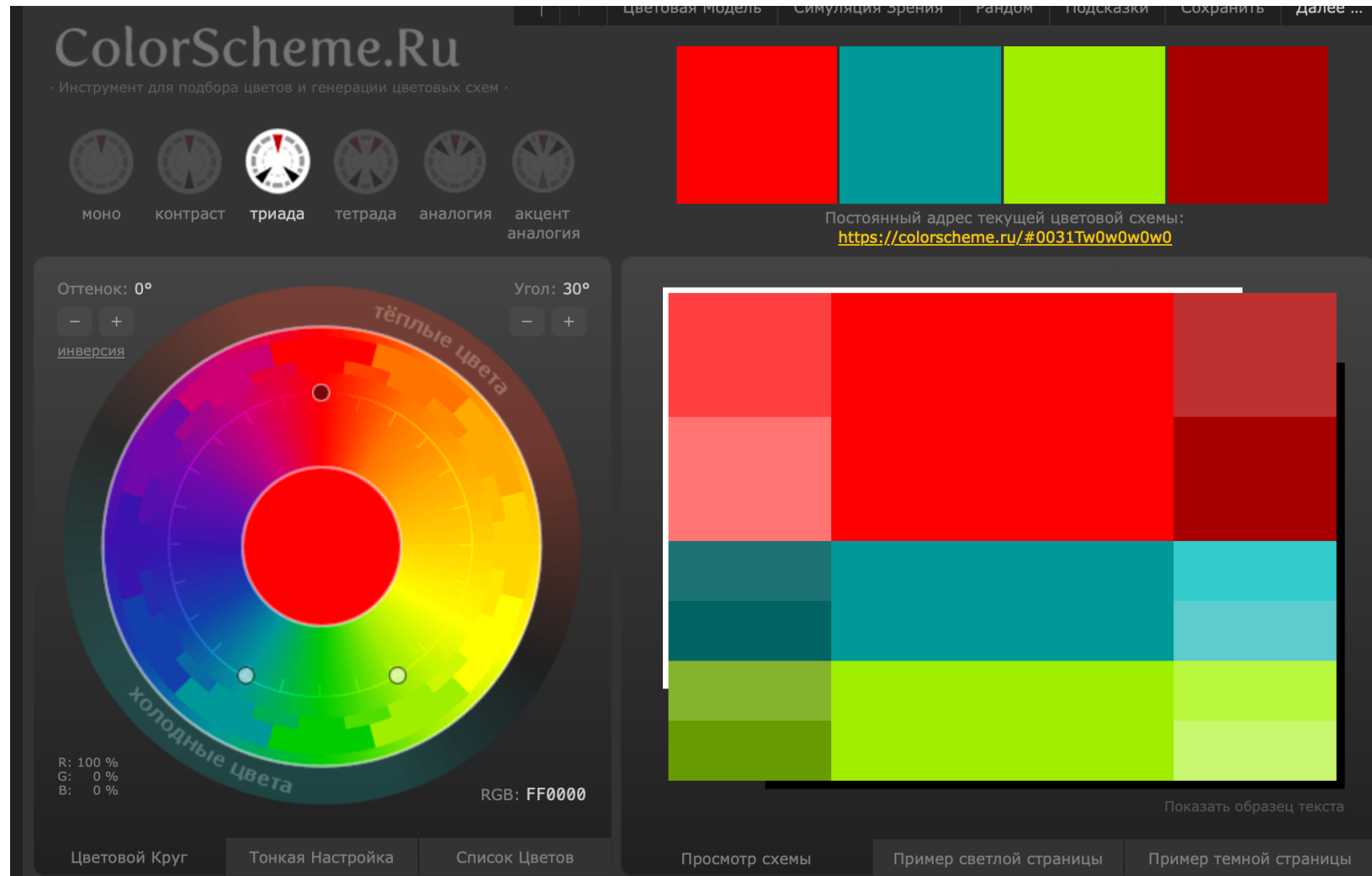
Figma/Pixso и CSS

- Дизайн первых 3 страниц приложения вы создаете в Figma/Pixso
- Затем стили ваших карточек вы переносите в CSS вашего проекта



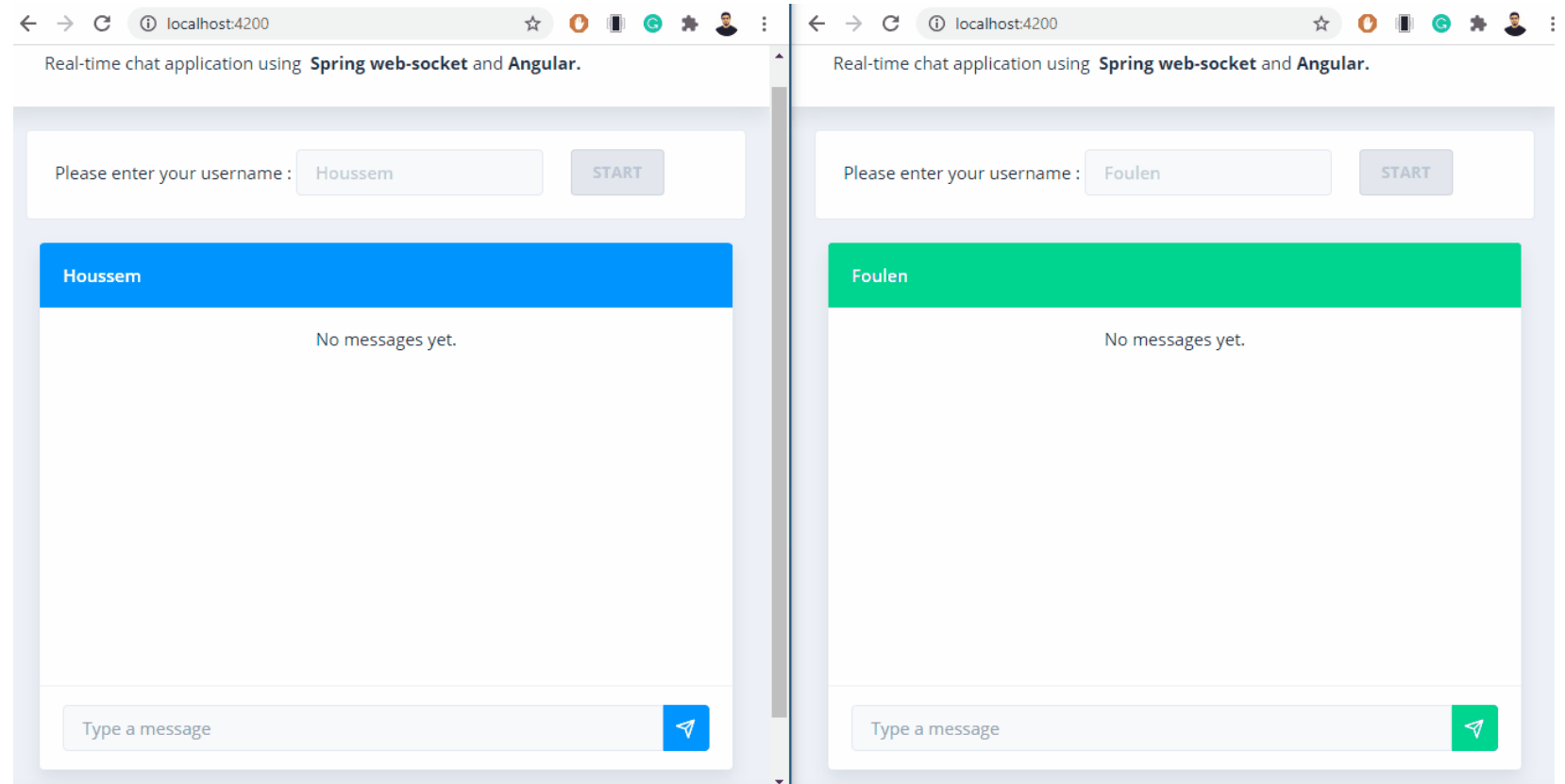
Дизайн приложения

- Работа над дизайном приложения с первого занятия
- Цветовая схема.
<https://colorscheme.ru>



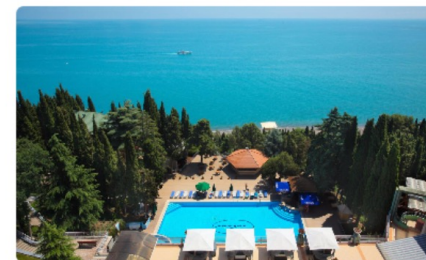
Real-time web

- Ajax
- Push
- WebSocket
- Подробнее
остановимся на
курсовой весной



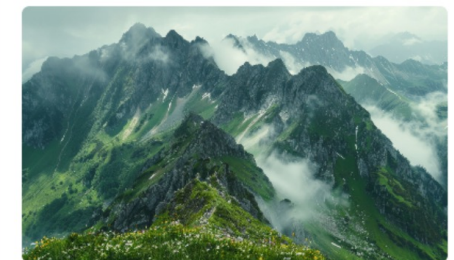
*Итоговое приложение курса

- Вам требуется разработать приложение для работы с услугами по вашей теме
- У всех один и тот же движок
- В этом примере услуги – это отели
- В первой лабораторной нужно реализовать три страницы: **лента**, **добавление** и **плитка карточек**
- Пока только просмотр, **редактирование** добавится позже



Отель 'Морской бриз'

Цена: 5000 руб.



Отель 'Горная вершина'

Цена: 3500 руб.

Web-фреймворки

- Клиентские фреймворки, **фронтенд SPA** (Angular, React, Vue)

Предназначены для разработки SPA. Реализуют концепцию «толстого» клиента и «тонкого» сервера. Основная функциональность реализована с использованием JavaScript/TS.

- Серверные фреймворки. Или **API бэкенд**, или **SSR**

Предназначены для разработки приложений на стороне веб-сервера. Реализуют концепцию «тонкого» клиента и «толстого» сервера (SSR). Используют традиционные языки веб-разработки: Python, PHP, Ruby, C#, Java, Go ...

Подразделяются на две категории:

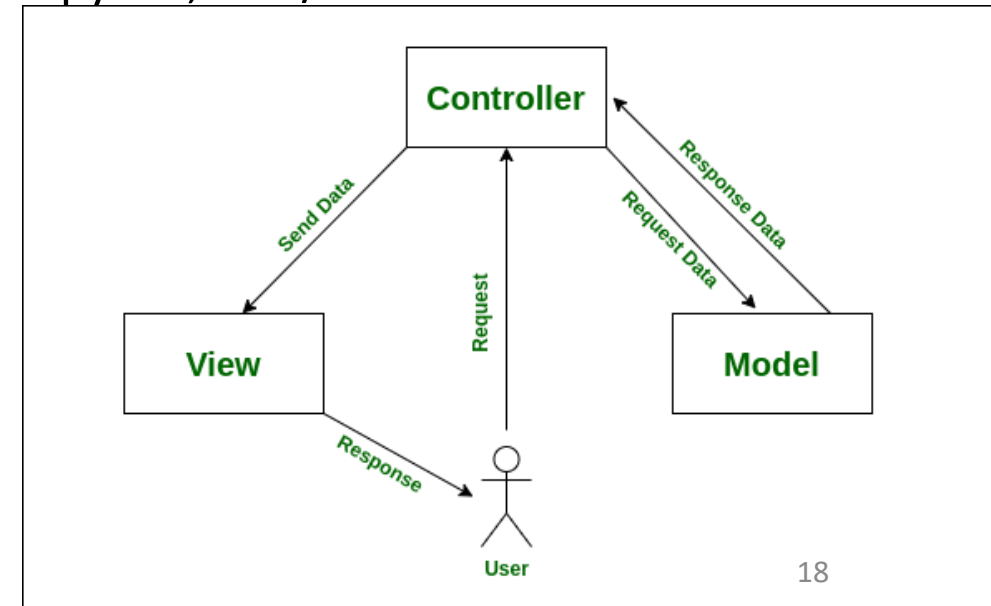
- Микрофреймворки (flask)
 - Традиционные фреймворки с полной функциональностью (.NET, Spring, Django, NestJS)
-
- В некоторых языках, созданных для web разработки (PHP и др), уже встроен шаблонизатор HTML
 - В отличие от таких языков, Python для веб-разработки обязательно нужны фреймворки (Django, flask и др). Например для Apache разработан модуль Apache mod_wsgi

*Традиционный серверный MVC фреймворк

- Статические файлы (статические HTML-документы, CSS, изображения, сценарии JavaScript и т.д.).
- Контроллеры (обработчики событий пользовательских действий).
- Модели (взаимодействие с БД).
- Представления (view). Шаблоны, генерирующие HTML-страницы и другое динамическое содержимое.

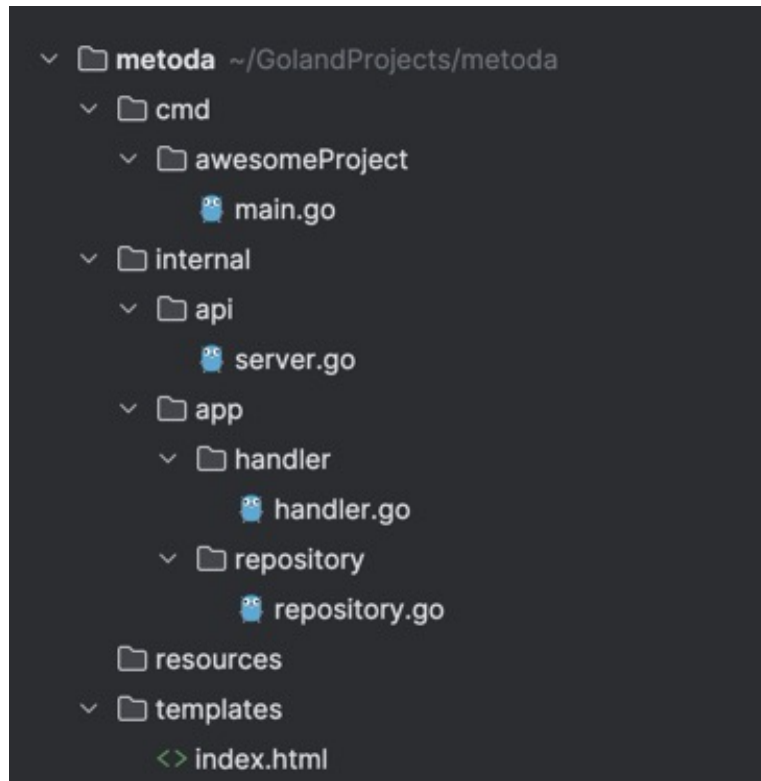
Конфигурирование фреймворка: действия при запуске приложения, конфигурирование пользовательских сессий, переписывание URL (привязка URL к контроллерам), безопасность (аутентификация и авторизация), кэширование, балансировка нагрузки, IOC / DI.

- Утилиты командной строки для управления фреймворком.
 - Скаффолдинг (создание структуры проекта, генерация кода , генерация готовых форм ввода и редактирования данных во время работы приложения).
 - Миграции (изменение структуры базы данных на основе моделей).



SSR проект на Golang

- На Golang у нас нет такого богатого web фреймворка, но все остается похожим: обработчики, шаблонизатор
- Обязательно сами формируем структуру проекта!



```
func (h *Handler) GetOrder(ctx *gin.Context) {
    idStr := ctx.Param("id") // получаем id заказа из
    // через двоеточие мы указываем параметры, которые
    id, err := strconv.Atoi(idStr) // так как функция
    if err != nil {
        logrus.Error(err)
    }

    order, err := h.Repository.GetOrder(id)
    if err != nil {
        logrus.Error(err)
    }

    ctx.HTML(http.StatusOK, "order.html", gin.H{
        "order": order,
    })
}
```

*Шаблонизация и коллекция Golang

- В Go нет классов, но есть структуры
- В этом примере модель-массив услуг используется в нескольких страницах-шаблонах

```
<html lang="en">
<header>
  <h1>
    <a href="/hello">Список</a>
  <!--Простой хедер, чтобы возвращаться на главную страницу-->
  </h1>
</header>
<h1>
  {{ .time }}
</h1>
<ul>
  {{ range .orders }}
  <li>
    <a href="/order/{{ .ID }}"> {{ .Title }} </a>
  </li>
  {{ end }}
</ul>
</html>
```

```
type Order struct { // вот наша новая структура
  ID    int // поля структур, которые передаются в шаблон
  Title string // ОБЯЗАТЕЛЬНО должны быть написаны с заглавной
}

func (r *Repository) GetOrders() ([]Order, error) {
  // имитируем работу с БД. Типа мы выполнили sql запрос
  orders := []Order{ // массив элементов из наших строк
    {
      ID:    1,
      Title: "first order",
    },
    {
      ID:    2,
      Title: "second order",
    },
    {
      ID:    3,
      Title: "third order",
    },
  }
  // обязательно проверяем ошибки, и если они появились
  // тут я снова искусственно обработаю "ошибку" чистого массива
  if len(orders) == 0 {
    return nil, fmt.Errorf("массив пустой")
  }

  return orders, nil
}
```

TypeScript (vs JS)

```
1 function sum(a: number, b: number): number {  
2     return a + b  
3 }
```

```
1 function sum(a, b) {  
2     return a + b  
3 }
```

```
1 interface Person {  
2     name: string;  
3     age: number;  
4 }  
5 function meet(person: Person) {  
6     return `Привет, я ${person.name}, мне ${person.age}`;  
7 }  
8 const user = { name: "Jane", age: 21 };  
9 console.log(meet(user));
```

Преимущества TypeScript*

TypeScript — язык программирования, представленный Microsoft в 2012 году. TypeScript является **обратно совместимым** с JavaScript и компилируется в последний. TypeScript отличается от JavaScript возможностью явного статического назначения типов, а также поддержкой подключения модулей

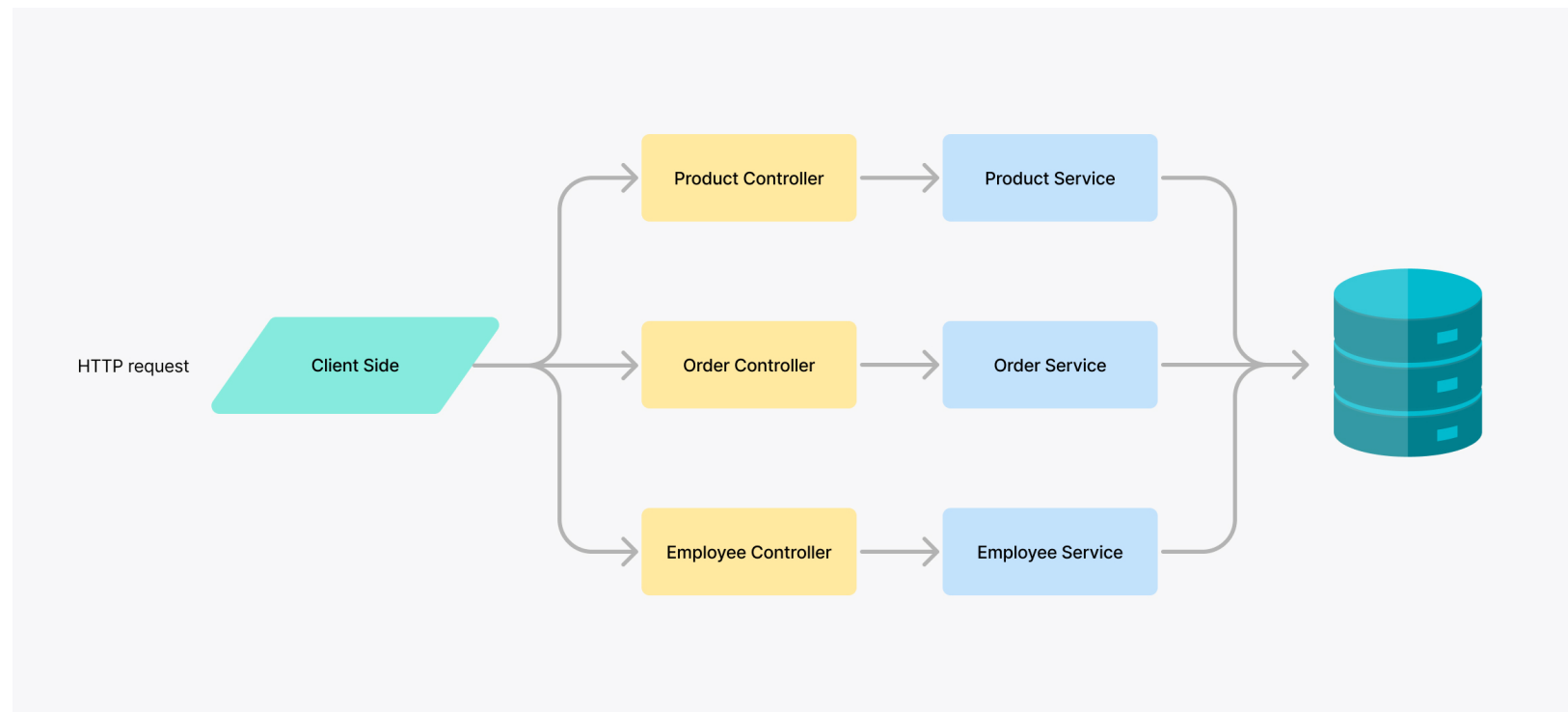
- Аннотации типов и проверка их согласования на этапе компиляции
- Интерфейсы, кортежи, декораторы свойств и методов, расширенные возможности ООП
- TypeScript — **надмножество JavaScript**, поэтому любой код на JavaScript будет выполнен и в TypeScript
- Поддержка ES6-модулей из коробки

Nest.js

[Nest.js](#) - это фреймворк для создания эффективных, расширяемых серверных приложений. **ОБЯЗАТЕЛЬНО** на TypeScript

Какие преимущества дает Nest.js

- готовую архитектуру;
- готовые решения для middleware;
- удобные механизмы валидации;
- возможность grpc;
- dependency injection
- java-подобное приложение.



* Контроллер – обработчики HTTP запросов

- Все данные у нас хранятся в массиве – это наша модель. Он хранится в оперативной памяти, поэтому мы в ЛР1 **не делаем** редактирование
- Для всех наших методов (комбинации url и HTTP метода) мы указываем код который будет выполняться
- Контроллер должен предоставляет данные через модель (сервис NestJS), но сейчас только массив

```
@Controller('bmstu-hello')
export class BmstuLabController {
  @Get()
  @Render('hello') // Рендерит views/hello.hbs
  getHello() {
    return {
      currentDate: new Date().toLocaleDateString(),
      list: [
        { name: 'NestJS', like: true },
        { name: 'Django', like: false },
        { name: 'HTML', like: true },
      ],
    };
  }
}
```

```
<ul>
  {{#each list}}
    {{#if this.like}}
      <li>We like {{ this.name }}!</li>
    {{else}}
      <li>We do not like {{ this.name }}.</li>
    {{/if}}
  {{/each}}
</ul>
```

Результат:

- We like NestJS!
- We do not like Django.
- We like HTML!

*Fast API, Jinja2

Тот же MVC: **данные** в массиве, по url определяем какой **шаблон** (представление) открыть

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Каталог</title>
</head>
<body>
    <h1>Каталог отелей</h1>

    <ul>
        {% for hotel in hotels %}
        <li>
            <h2>{{ hotel.title }}</h2>
            <p>Цена: {{ hotel.price }} руб.</p>
        </li>
        {% endfor %}
    </ul>
</body>
</html>
```

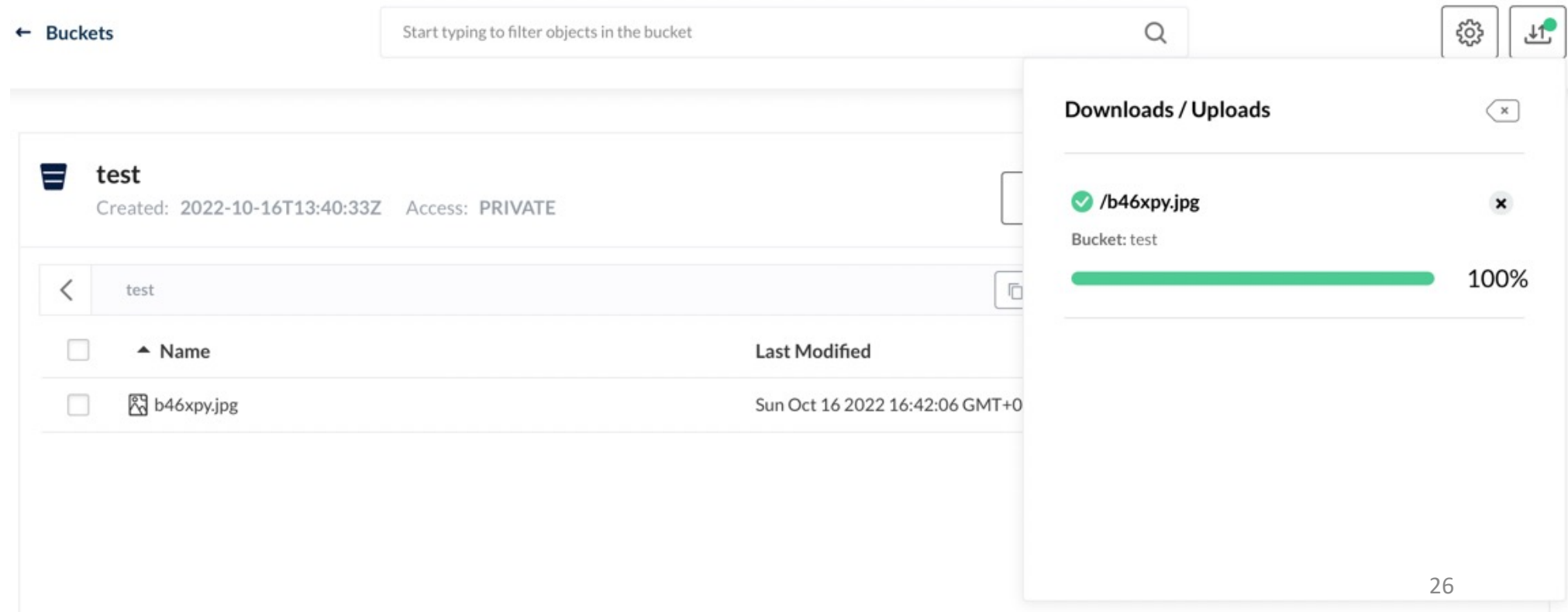
```
hotels_db = [
    {
        "id": 1,
        "title": "Отель 'Морской бриз'",
        "price": 5000,
        "description": "Прекрасный отель на берегу моря.",
        "image_url": "/static/img/hotel1.jpg",
        "video_url": "/static/img/hotel1_vid.mp4"
    },
    {
        "id": 2,
        "title": "Отель 'Горная вершина'",
        "price": 3500,
        "description": "Уютное шале в горах. Идеально для отдыха.",
        "image_url": "/static/img/mountains.jpg",
        "video_url": "/static/img/mountains.mp4"
    },
]
```

```
router = APIRouter()
templates = Jinja2Templates(directory="templates")

@router.get("/")
def get_catalog(request: Request):
    # Добавляем context явно
    return templates.TemplateResponse(
        request=request,
        name="index.html",
        context={"hotels": hotels_db}
    )
```

Minio

- Вам потребуется установить S3 хранилище Minio для ваших изображений (через Docker)
- Сейчас вы **вручную добавляете** изображения и используете их в своем приложении
- Позже в ЛР-3 ваше приложение будет само загружать изображения в S3



* Docker

- Разработчики пишут код и запускают его на своих ноутбуках
- Пользователи используют приложение, которое размещается в датацентре, совсем **на других** компьютерах с Linux
- Более того, мы не хотим чтобы разное ПО в датацентре **мешало** друг другу
- Докер – это следующий шаг после виртуальных машин
- Мы **изолировали** каждый контейнер, но для него требуются **минимальные ресурсы**

<https://github.com/iu5git/Web/tree/main/tutorials/git-docker>

	db-1 ff0166104e89	postgres:12	Running	0.08%	5432:5432	
	redis-1 cb3d585cf2af	redis:6.2-alpine	Running	0.2%	6379:6379	2 minutes ago
	minio be2a6974a216	minio/minio:latest	Running	0%	9000:9000 Show all ports (2)	0 seconds ago

- **Файл** – список команд, что запустить и какой версии
- Готовый **образ**, который можно запустить
- Образ можно запустить в контейнерах со своими данными
- **Volume** – данные которые сохраняться после его удаления

```
db:
  # название моего имеджа
  image: postgres:12 # скачает image по
  volumes:
    # часть настроек для хранения данны
    - type: volume
      source: postgresdb-data
      target: /var/lib/postgresql/data
  ports:
    # порты
    - "5432:5432"
```